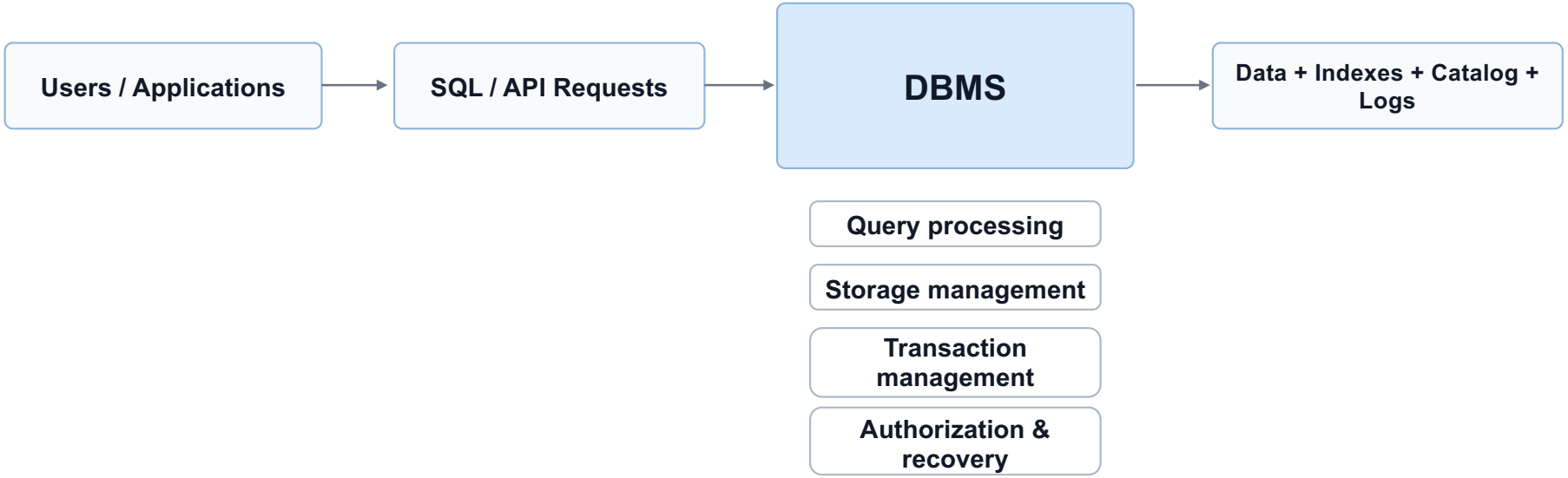
Course Information

- **Lecturer: PhD. Nguyen Thi Cam Van – vanntc@vnu.edu.vn**
- **DSKT Lab - Faculty of Information Technology**
- **Office: 201-E3**
- **Learning channels: LMS (portal)**
- **Study materials:** Slides + Books
  - [SKS19] A. Silberschatz, H. F. Korth, S. Sudarshan, *Database System Concepts, 7th Edition*, McGraw-Hill, 2019.

Database Management Systems

Lecture 1 : Inside a Database Management System



A DBMS is not just a data file. It is a running software system that stores, retrieves, protects, recovers and optimizes access to data.

Main sources: Silberschatz, Korth & Sudarshan, Database System Concepts, 7e; MySQL 8.4 Reference Manual

Lecture 1 Learning objectives

- **Distinguish database, DBMS, and database system**.
  - database = organized data; DBMS = software for managing databases; database system = DBMS plus applications, users, storage and environment
- **Explain the main DBMS components.**
  - query processor, storage manager, transaction manager, recovery manager, authorization/integrity manager, data dictionary
- **Trace one SQL statement through the system.**
  - parse/validate → choose plan → execute → access pages/indexes → enforce rules
- **Inspect a MySQL server at a basic administrative level.**
  - version, databases, tables, engines, sessions/processes, variables, metadata

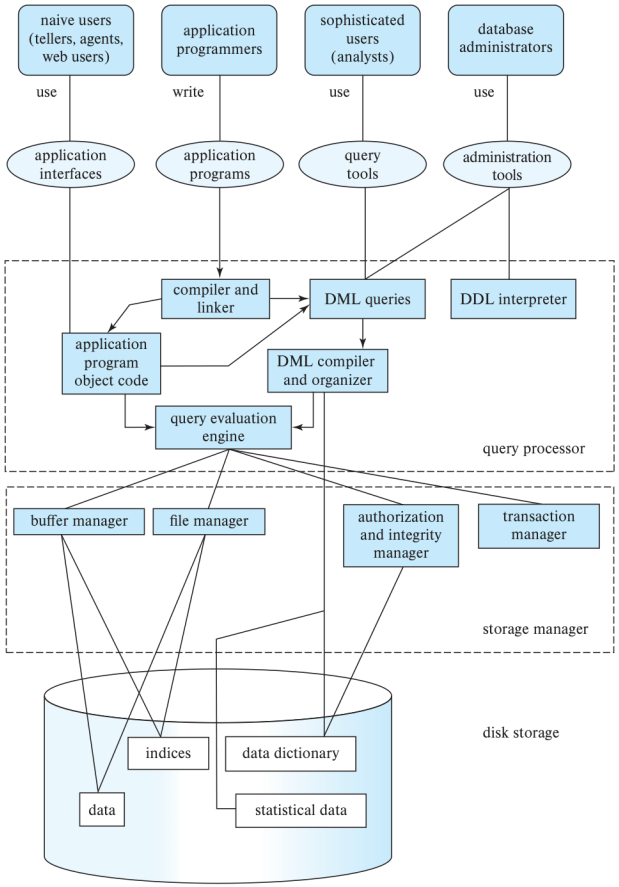


Figure 1.3 System structure.

[SKS 7e, Ch.1, pp.18–24]

What students already know and what this course adds

Basic Database course	This DBMS course
Relational model	DBMS architecture and engine components
SQL query writing	How SQL is processed and optimized
Tables, keys, constraints	How data is stored, indexed and accessed
Basic transactions	Isolation, concurrency and recovery
Application-level use	Administration, security, backup, monitoring

```
SELECT c.name, SUM(o.total)
FROM customers c JOIN orders o
  ON c.id = o.customer_id
WHERE o.created_at >= '2026-01-01'
GROUP BY c.name;
```

- Writing the SQL is only the starting point.
- The DBMS still has to validate the query, choose an execution plan, read data efficiently, and preserve correctness.
- This course studies those mechanisms from the perspective of system understanding and administration.

[SKS 7e, Ch.1; Ch.15–16]

Course structure: 12 teaching blocks

Block	Core problem	Hands-on connection
L1 Inside DBMS	What happens after SQL reaches the DBMS?	Install/inspect MySQL
L2 DB programmability	What logic should live inside the DBMS?	View, procedure, function, trigger
L3 Storage	Where is data stored and how much storage is needed?	Table/index size; estimation
L4 Indexing	Which index helps this workload and what does it cost?	Index experiments
L5 Query processing	Why did the DBMS choose this plan?	EXPLAIN / EXPLAIN ANALYZE
L6+L7 Transactions + concurrency	How is correctness preserved under concurrent access?	Two-session anomalies/locks/deadlocks
L8+L9+L10 Administration + scale	How do we secure, recover, partition and replicate?	Privileges, backup, partitioning, replication
L11+L12 Tuning + modern DB	How to diagnose bottlenecks and choose a DBMS?	Sick database + technology selection

Teaching blocks can be stretched across 15 weeks using lab, assessment and project-clinic sessions.

Course syllabus; academic backbone: [SKS 7e] and [MySQL 8.4 Reference Manual]

Assessment and project expectations

- **40% in-class assessments**
  - 2-3 short assessments at the end of selected lectures
  - questions may combine theory, plan reading, SQL interpretation and scenario analysis
- **60% course project**
  - starts from Lecture 2/3
  - students analyze a real or realistic database problem
  - must include data, workload, baseline, evidence, and trade-off analysis

A good project starts with a problem, not with a product name.

Weak topic	Better topic
“Study MongoDB”	Analyze whether document storage fits a changing product-catalog workload.
“Build a sales website”	Identify slow queries in a sales workload and evaluate indexing/partitioning.
“Compare MySQL and PostgreSQL”	Choose a DBMS for a workload and justify with requirements and evidence.

Project design aligned with [SKS 7e, Ch.1, Ch.25]

Today’s key question

When a SQL statement reaches a DBMS, what components handle it and what responsibilities do they have?

- **Part A: From database to database system**
  - purpose of a DBMS
  - data abstraction
  - languages and users
- **Part B: Database engine**
  - query processor
  - storage manager
  - transaction and recovery
- **Part C: MySQL as our running system**
  - storage engines
  - metadata
  - server inspection
- **Part D: Lab 0**
  - install/load sample database
  - inspect a running DBMS

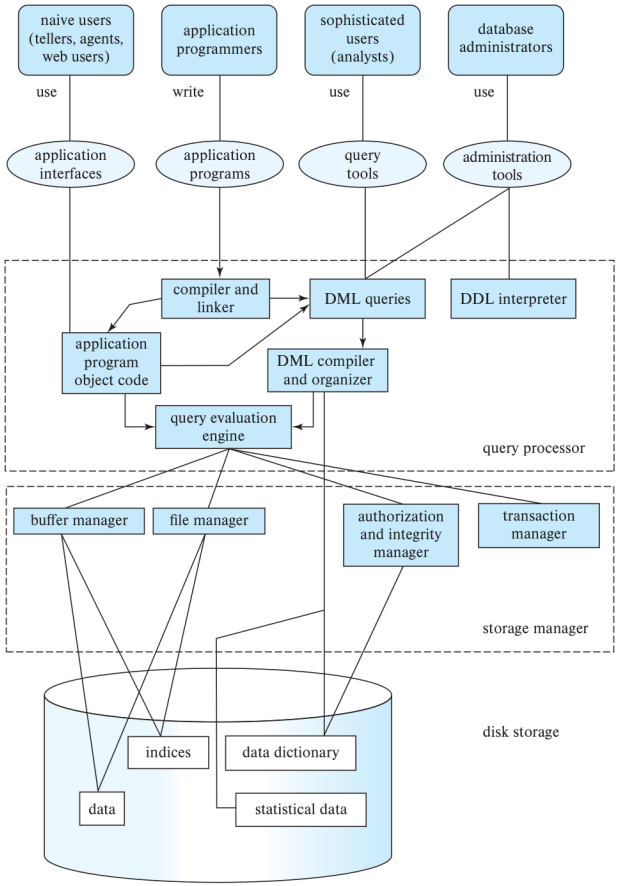
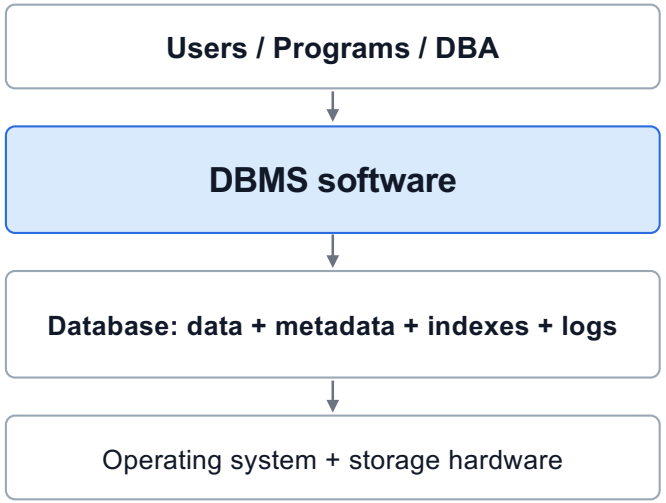


Figure 1.3 System structure.

[SKS 7e, Ch.1, §1.6–1.8]

Database, DBMS, and database system

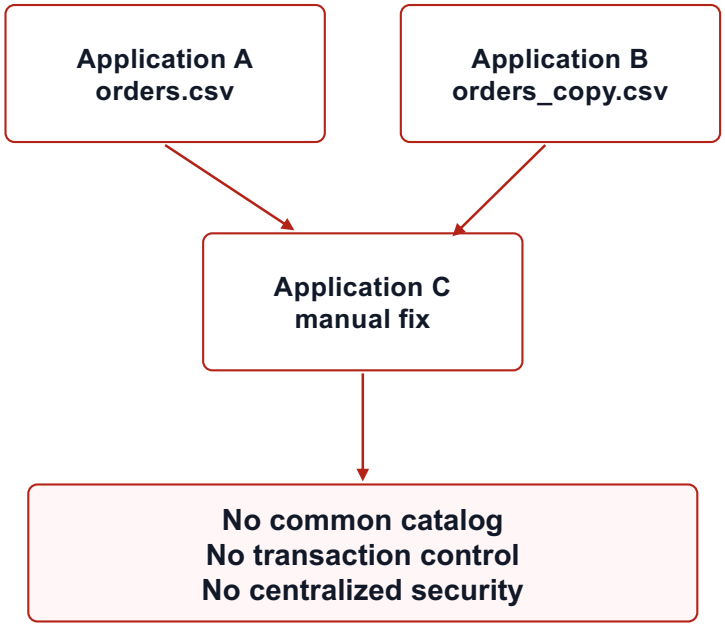
Term	Meaning	Example
Database	A collection of interrelated data.	tables, records, indexes, catalog information
DBMS	Software that manages databases and provides controlled access to them.	MySQL, PostgreSQL, Oracle Database, SQL Server
Database system	DBMS plus databases, applications, users, administrators, and hardware/storage environment.	- a university registration - banking information system



[SKS 7e, Ch.1, §1.1–1.2]

Why database systems exist: limitations of file processing

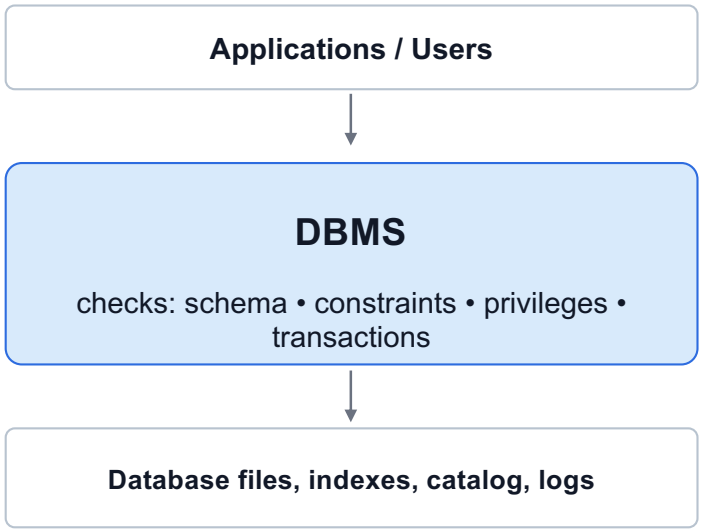
- **File-processing systems make applications manage too much.**
  - each application decides file format, validation, access control and recovery
  - logic is duplicated and often inconsistent across programs
- **Typical problems**
  - data redundancy and inconsistency
  - difficulty in accessing data
  - data isolation across files and formats
  - integrity and atomicity problems
  - concurrent-access anomalies
  - security problems
- **DBMS objective**
  - provide a systematic, controlled way to **store**, **retrieve** and **modify** data



[SKS 7e, Ch.1, §1.2]

The purpose of a DBMS: shared control over data

- **A DBMS controls data access through a common interface.**
  – applications send queries/updates instead of directly manipulating files
  – the DBMS checks metadata, constraints, privileges and transaction state
- **A DBMS supports both efficiency and correctness.**
  – **efficiency**: *storage organization, indexing, query optimization, buffering*
  – **correctness**: *integrity constraints, transactions, concurrency control, recovery*
- **Administration becomes part of the system.**
  – backup/recovery, user management, monitoring and configuration are DBMS-level responsibilities



[SKS 7e, Ch.1, §1.2; §1.6]

Data abstraction: users should not manage low-level storage

- **Physical level**
  - describes how data are actually stored
  - includes low-level structures that ordinary users should not need to know
- **Logical level**
  - describes what data are stored and what relationships exist
  - used by database administrators and designers
- **View level**
  - shows only part of the database to a user or application
  - simplifies interaction and can support access control

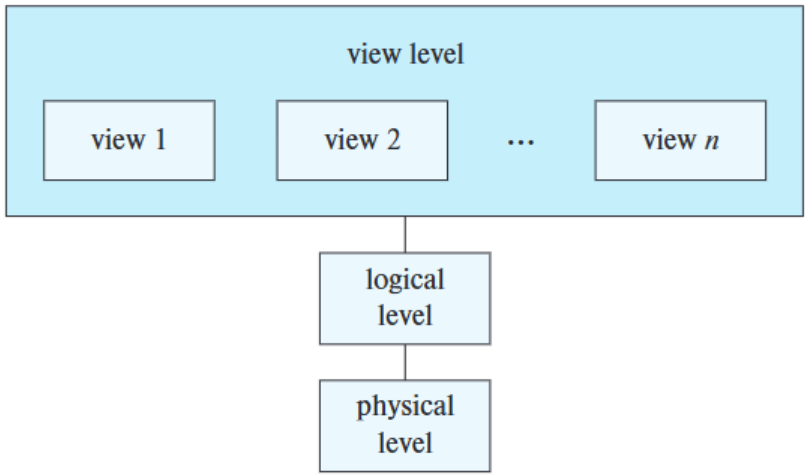


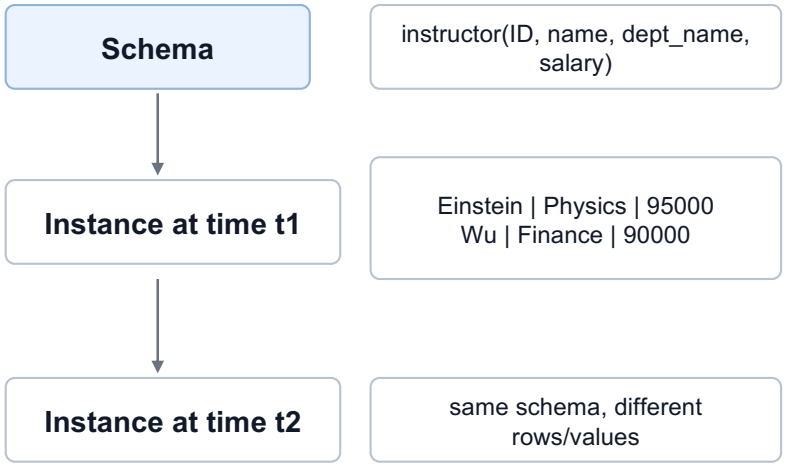
Figure 1.2 The three levels of data abstraction.

Key term: physical data independence = users can be shielded from changes in physical storage details.

[SKS 7e, Ch.1, §1.3.3, pp.9–11]

Schemas and instances

- **Schema**
  – the logical design of the database
  – specifies relations/tables, attributes, data types, constraints and relationships
  – changes relatively infrequently
- **Instance**
  – the actual content of the database at a particular point in time
  – **changes whenever data are inserted, updated or deleted**
- **Why this matters**
  – DBMS components use schema information for validation, optimization, integrity checking and authorization



[SKS 7e, Ch.1, §1.3.4, pp.12–13]

Relational model

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Figure 1.1 A sample relational database.

- **A relation is represented as a table.**
  - rows represent records/tuples
  - columns represent attributes
  - each column has a name and a domain/type
- **Relationships can be represented by common attribute values**.
  - in the sample, dept_name connects instructor with department
  - foreign keys and constraints are introduced in prior database courses and used throughout DBMS operation
- **In this course, the same logical table becomes a physical and operational object.**
  - stored in pages/files
  - indexed for access
  - protected by privileges
  - updated under transactions

[SKS 7e, Ch.1, §1.3.2, pp.9–10]

Database languages: DDL, DML, TCL, DCL

Class	Purpose	Examples	DBMS component touched
DDL (Data Definition Language)	Define database schema and objects.	CREATE TABLE, ALTER TABLE, DROP	catalog / DDL interpreter
DML (Data Manipulation Language)	Query and modify data.	SELECT, INSERT, UPDATE, DELETE	query processor / storage manager
TCL (Transaction Control Language)	Control transaction boundaries.	COMMIT, ROLLBACK, SAVEPOINT	transaction manager / recovery
DCL (Data Control Language)	Control access privileges.	GRANT, REVOKE	authorization manager

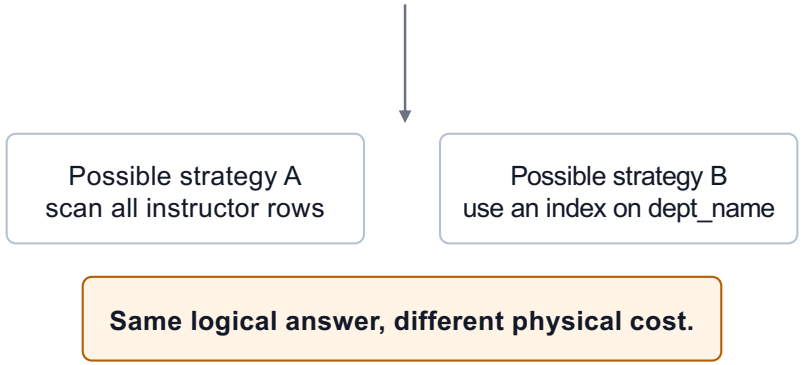
SQL statements are processed by different DBMS subsystems depending on their purpose.

[SKS 7e, Ch.1, §1.4; SKS 7e, Ch.4, §4.3 and §4.7]

SQL is declarative: “what” rather than “how”

- **A SQL query specifies the desired result.**
  – it normally does not specify which index to use
  – it does not tell the DBMS exactly which join algorithm to execute
  – it does not specify which disk pages to read first
- **The DBMS must choose an efficient execution strategy.**
  – translate SQL to an internal representation
  – apply equivalence transformations
  – estimate cost using statistics and physical information

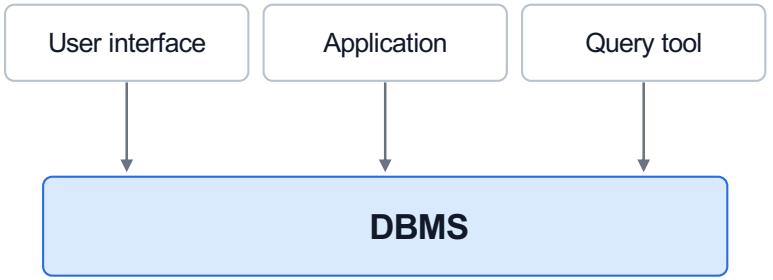
```
SELECT name
FROM instructor
WHERE dept_name = 'Physics';
```



[SKS 7e, Ch.1, §1.4.2; Ch.15–16]

Database users and administrators

- **Naive users**
  - interact through predefined application interfaces
  - do not write SQL directly
- **Application programmers**
  - write applications that access the database
  - use SQL/API calls and application logic
- **Sophisticated users**
  - use query tools directly
  - analysts and power users
- **Database administrators**
  - define authorization, monitor performance, backup/recover data, manage storage and availability



This course emphasizes the transition from SQL user to database-system analyst/administrator.

Application architecture: where is the database?

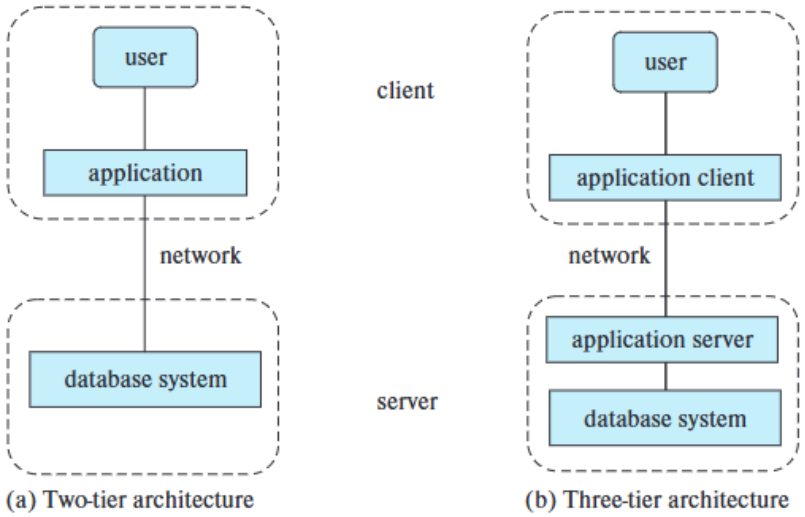


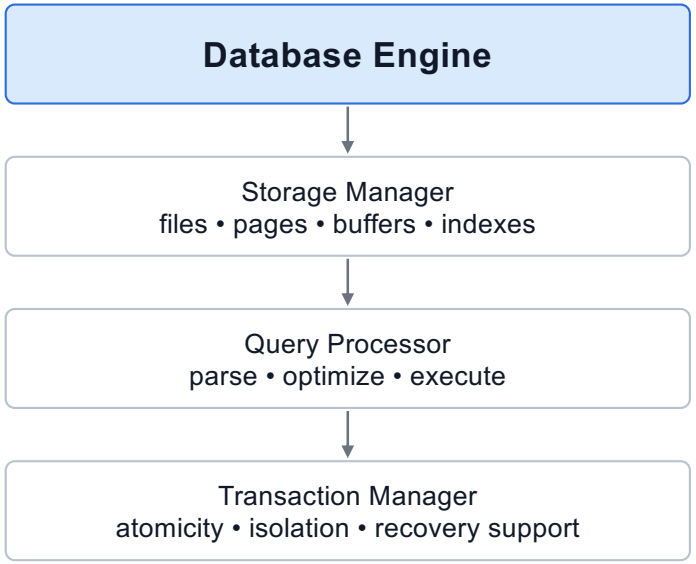
Figure 1.4 Two-tier and three-tier architectures.

- **Two-tier architecture**
  - application directly invokes database system functionality
  - common in earlier-generation client/server applications
- **Three-tier architecture**
  - front-end talks to an application server
  - application server talks to the database system
  - business logic is typically placed in the application server
- **Administration implication**
  - DBMS security, connection management and workload characteristics depend on the surrounding application architecture

[SKS 7e, Ch.1, §1.7, pp.22–23]

Database engine: three major responsibility areas

- **Storage management**
  - organize data and indexes on storage
  - move data between disk and main memory as needed
- **Query processing**
  - translate high-level queries into efficient low-level operations
  - choose among alternative execution strategies
- **Transaction management**
  - treat a sequence of operations as one unit
  - support atomicity, isolation and durability



[SKS 7e, Ch.1, §1.6, pp.18–21]

Overall database-system structure

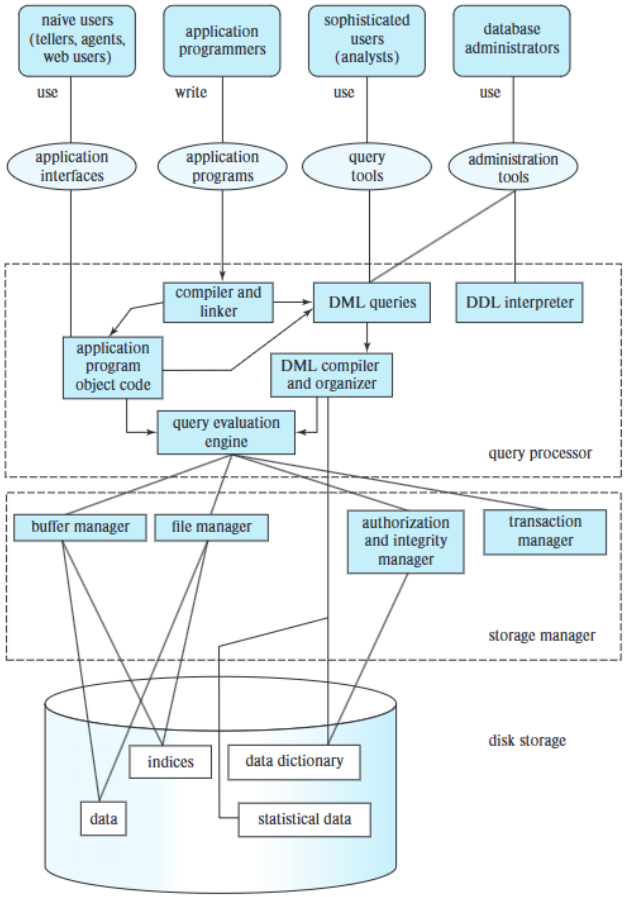


Figure 1.3 System structure.

- **Top part: users and tools**
  - applications, query tools and administration tools send requests to the system
- **Middle part: query processor**
  - handles DML/DDL processing and query evaluation
- **Lower part: storage manager**
  - includes buffer, file, authorization/integrity and transaction management
- **Disk storage**
  - contains data, indexes, data dictionary and statistical data

[SKS 7e, Ch.1, Fig.1.3, p.22]

Overall database-system structure

Request entry

• A user/application submits DML, DDL or administrative commands.

• The command is not executed directly on files.

• It passes through DBMS control layers.

Query processing

• DML queries are compiled and organized.

• The query evaluation engine executes a chosen plan.

• The optimizer will be studied in L5.

Storage + control

• The storage manager accesses data, indexes and catalog information.

• Authorization, integrity, transaction and recovery concerns affect execution.

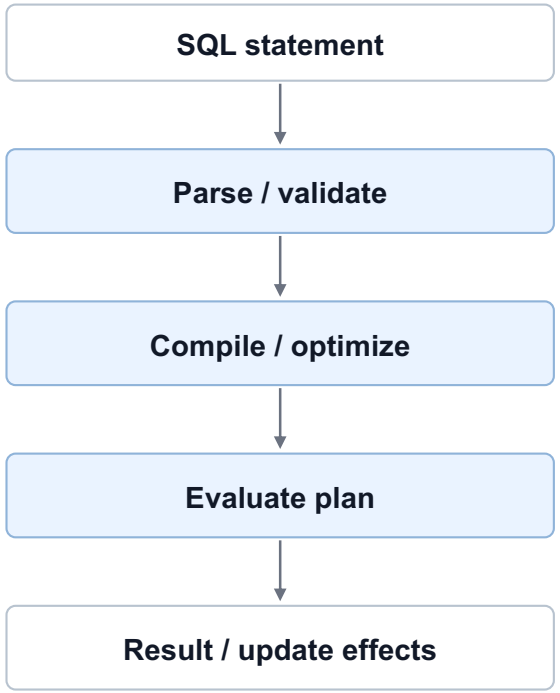
• Storage and indexing are studied in L3–L4.



[SKS 7e, Ch.1, Fig.1.3, p.22]

Query processor: from statement to execution

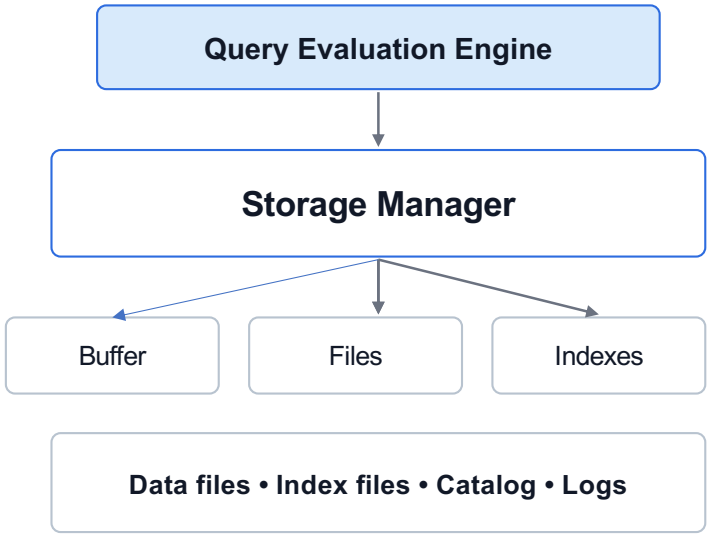
- **DDL interpreter**
  - interprets schema-definition statements
  - records definitions in the data dictionary/catalog
- **DML compiler and organizer**
  - translates DML statements into an evaluation plan
  - uses information about schema, indexes and statistics
- **Query evaluation engine**
  - executes low-level instructions generated for the plan
  - requests pages/records through storage components



[SKS 7e, Ch.1, §1.6.2; Ch.15–16]

Storage manager: controlled access to physical data

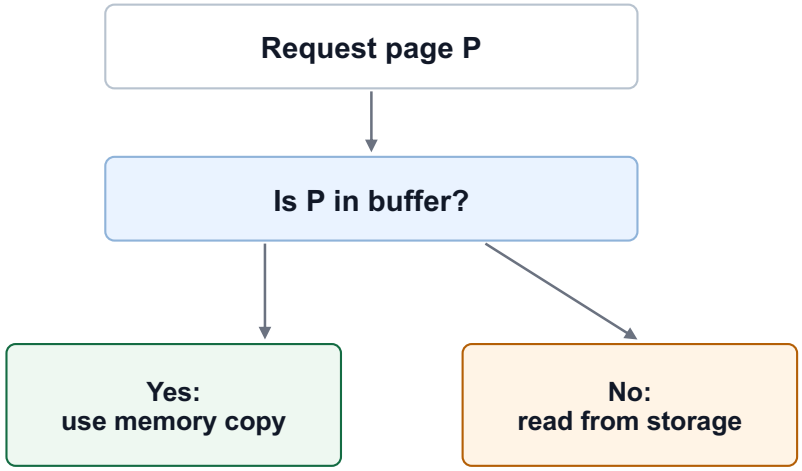
• The storage manager provides the interface between low-level stored data and queries/applications.
  – data are stored persistently; pages and records are moved into memory as needed
  – the query engine should not manipulate OS files directly
• **Core responsibilities introduced in this lecture**
  – file manager and disk-space management
  – buffer manager
  – data dictionary and statistical data
  – authorization/integrity and transaction-related coordination
• Later lectures open this box.
  – L3: physical storage and sizing
  – L4: indexing and access paths
  – L11: monitoring/tuning



[SKS 7e, Ch.1, §1.6.1; Ch.13]

Buffer manager: the first performance intuition

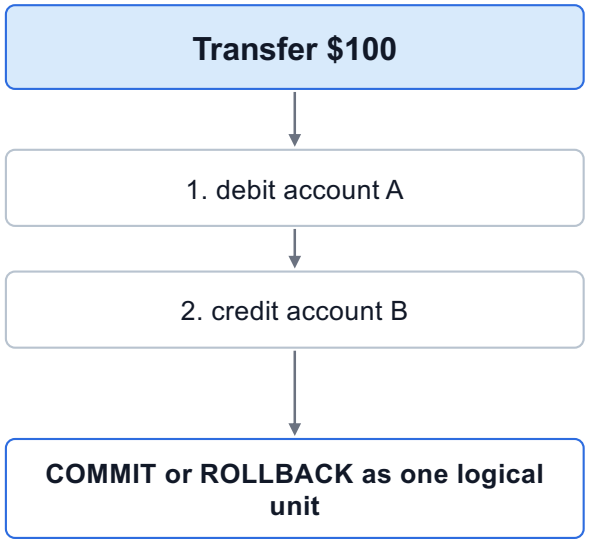
- **Databases are usually larger than main memory.**
  – not all data can be kept in RAM
  – persistent storage is durable but slower to access than memory
- **The buffer manager decides what pages are in main memory.**
  – if a needed page is already in the buffer, access is faster
  – if not, the system must fetch it from storage



[SKS 7e, Ch.1, §1.6.1; Ch.13, §13.5]

Transaction manager: treating several operations as one unit

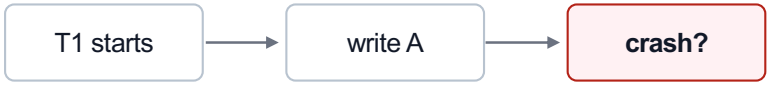
- **A transaction is a unit of program execution that accesses and possibly updates data.**
  - examples: money transfer, order creation, ticket reservation
  - a transaction may contain multiple reads and writes
- **The system must support atomicity and isolation.**
  - atomicity: either all effects occur or none do
  - isolation: concurrent transactions should not interfere in unacceptable ways
- **L1 scope**
  - define the responsibility only
  - formal transaction behavior is studied in L6 and L7



[SKS 7e, Ch.1, §1.6.3; Ch.17]

Recovery manager: failure must not leave partial effects

- **Failures are normal events in database systems.**
  – application error, transaction abort, system crash, storage failure
  – a correct DBMS must define what database state remains after failure
- **Recovery supports atomicity and durability.**
  – a failed transaction should not partially change the database
  – a committed transaction should survive relevant failures
- **Log-based recovery is introduced later.**
  – L8/L9 studies logs, backup and recovery in detail
  – MySQL implementation uses binary logs and InnoDB logs depending on context



DBMS must decide: undo partial effects or redo committed effects according to transaction status and logs.

[SKS 7e, Ch.1, §1.6.3; Ch.19]

Authorization and integrity: every operation is checked

- **Authorization controls who can perform which operations.**
  – users/roles may have different privileges
  – not every user should be allowed to read, update, delete or define objects
- **Integrity constraints preserve valid database states.**
  – data types, keys, foreign keys, checks and other constraints
  – the DBMS rejects operations that violate declared constraints
- Why this matters?
  – query execution is not only a performance problem
  – security and correctness checks are part of DBMS behavior

```
DELETE FROM salary;
```

Before execution, the DBMS should ask:

- Who is the current user?
- Does the user have DELETE privilege?
- Do constraints/triggers affect this update?
- What if this statement runs inside a transaction?

[SKS 7e, Ch.1, §1.6.1; Ch.4, §4.7; MySQL 8.4 Manual, Ch.8]

Catalog / data dictionary: metadata used by the DBMS

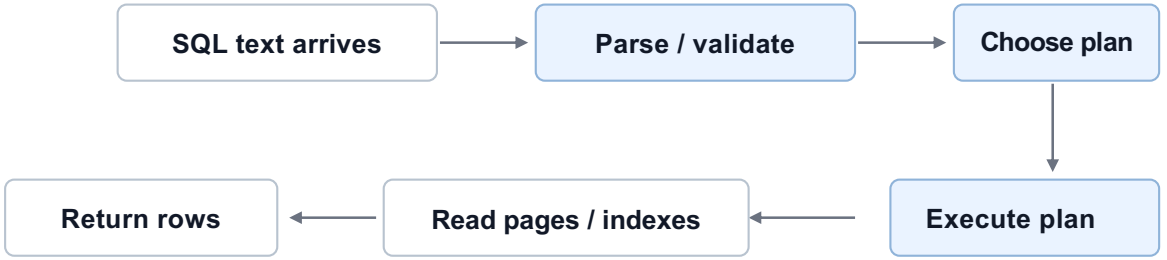
- **The data dictionary stores metadata.**
  - relation/table names, attributes, types and constraints
  - users/privileges in product-specific catalogs
  - indexes and statistical information used by the optimizer
- **The DBMS consults metadata frequently.**
  - validation: does the table/column exist?
  - authorization: is the operation permitted?
  - optimization: what indexes/statistics are available?
- **MySQL exposes metadata through INFORMATION_SCHEMA and other system schemas.**
  - we inspect these in Lab 0

Metadata question	Example source
What tables exist?	INFORMATION_SCHEMA.TABLES
What columns exist?	INFORMATION_SCHEMA.COLUMNS
What indexes exist?	SHOW INDEX / INFORMATION_SCHEMA.STATISTICS
What engines are used?	SHOW ENGINES / TABLES.ENGINE
What sessions are active?	SHOW PROCESSLIST / Performance Schema

[SKS 7e, Ch.1, Fig.1.3; Ch.13, §13.4; MySQL 8.4 Manual, INFORMATION_SCHEMA]

Follow one SQL query through the DBMS

```
SELECT *
FROM orders
WHERE customer_id = 1729;
```



- **The query looks simple, but several subsystems are involved.**
  - parser/validator use schema/catalog information
  - optimizer may choose a table scan or index access
  - executor requests data through storage and buffer components
  - authorization, transactions and logging can affect execution
- **L1 goal: understand the big picture, not every algorithm.**
  - details are covered in later lectures

[SKS 7e, Ch.1, Fig.1.3; Ch.15, §15.1]

Step 1 — parse and validate

- **Parsing checks whether the SQL is syntactically valid.**
  – SQL grammar, statement structure and keywords must be valid
  – invalid SQL fails before optimization or execution
- **Validation checks the database context.**
  – referenced tables and columns must exist
  – data types must support the requested operations
  – the user must have the required privileges
- **Output of this phase**
  – an internal representation that can be optimized and executed

```
SELECT * FROM orders
WHERE customer_id = 1729;
```

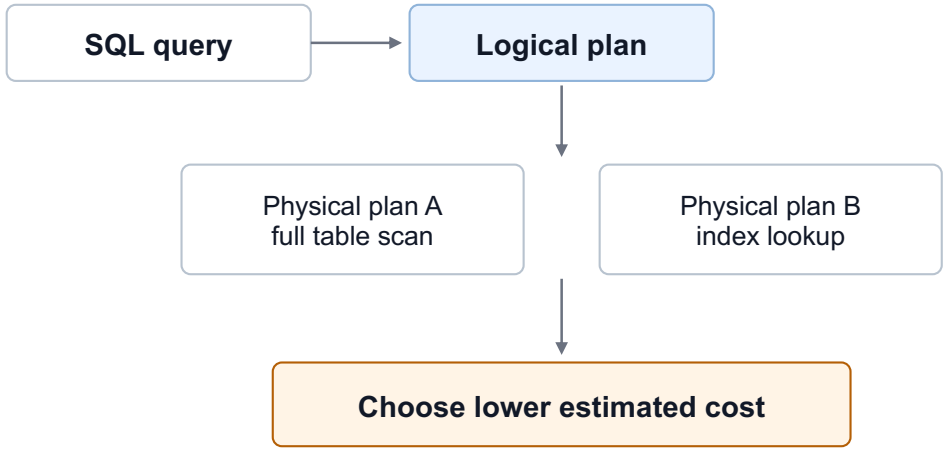
Catalog checks

- Does table orders exist?
- Does column customer_id exist?
- Is comparison with 1729 type-compatible?
- Can this user SELECT from orders?

[SKS 7e, Ch.1, §1.6.2; Ch.15, §15.1]

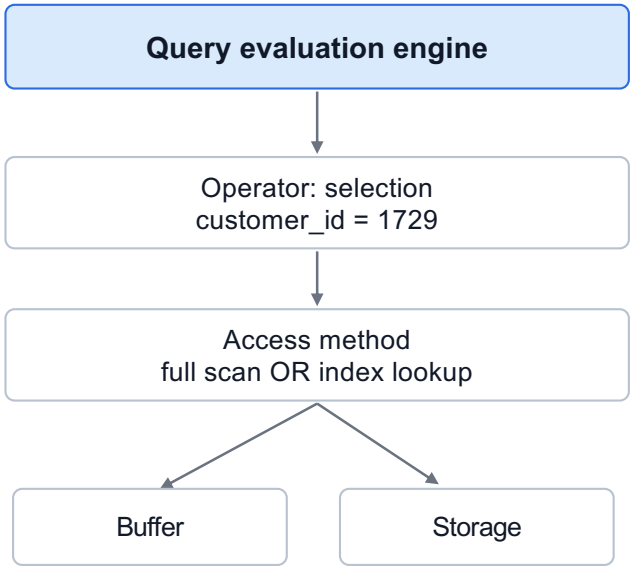
Step 2 — choose a logical and physical plan

- **Logical plan**
  – represents the query using relational operations
  – different but equivalent expressions can produce the same answer
- **Physical plan**
  – specifies how operations are implemented
  – examples: full scan, index scan, nested-loop join, sort, hash join
- **Cost-based optimization**
  – DBMS estimates the cost of alternatives using statistics and physical information
  – chosen plan should be efficient for the current data/workload assumptions



Step 3 — execute the plan and access storage

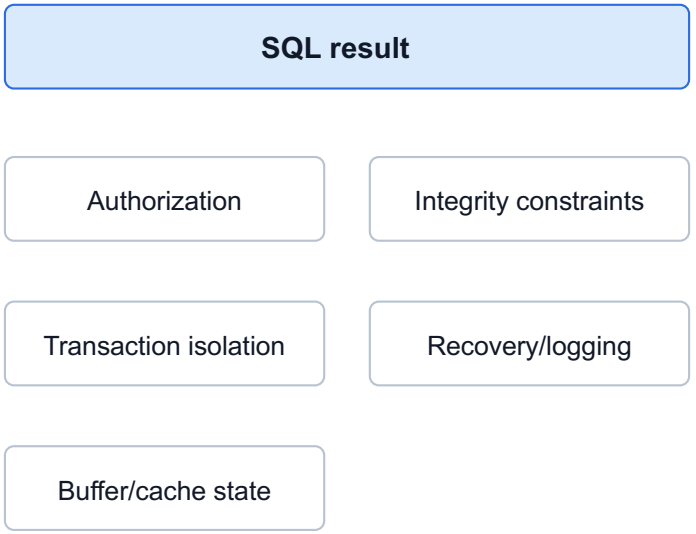
- **The query evaluation engine executes the selected plan.**
  - operators produce and consume tuples/records
  - operators request access to tables and indexes
- **Storage access is mediated by DBMS components.**
  - buffer manager may serve pages from memory
  - file manager accesses persistent data and index structures
  - statistics and catalog information may influence execution choices
- **Result production**
  - the DBMS returns qualifying rows to the client or application



[SKS 7e, Ch.1, §1.6.1–1.6.2; Ch.15]

Step 4 — preserve correctness while returning results

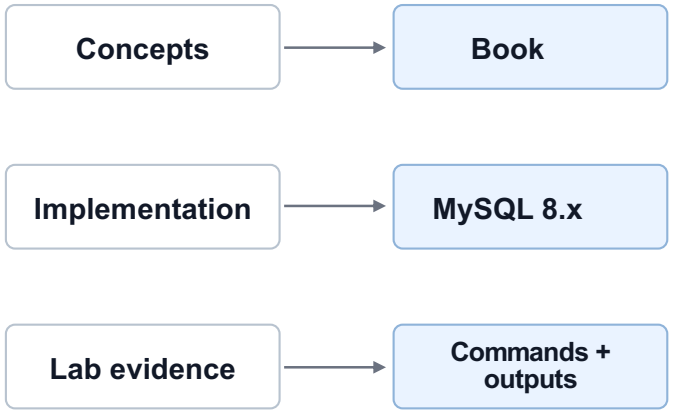
- **A DBMS executes queries in a controlled environment.**
  - permissions are checked
  - constraints may be enforced
  - transactions may define what state the query is allowed to see
  - logs may be generated for updates
- **Even read-only queries are not isolated from system concerns.**
  - concurrent updates may affect visibility depending on isolation level
  - buffer/cache state affects observed performance
  - statistics may affect plan choice
- **Main lesson**
  - a SQL statement is a request to an entire database system



[SKS 7e, Ch.1, §1.6; Ch.4, Ch.17–19]

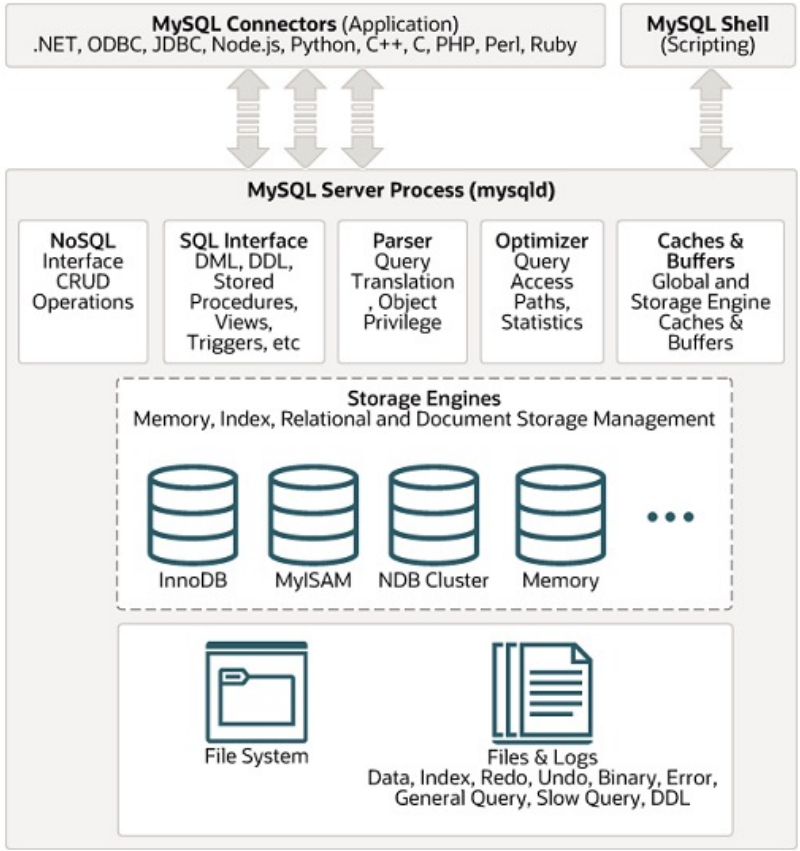
MySQL as the running DBMS for this course

- **We use MySQL 8.x for hands-on work in the first offering.**
  – existing course labs and datasets are already MySQL-based
  – MySQL supports the required teaching topics: SQL, views, stored routines, triggers, indexes, transactions, privileges, backup, replication and partitioning
- **Theory remains DBMS-general.**
  – the book explains concepts independently of a specific product
  – MySQL manual is used only for product-specific syntax and behavior
- **Modern comparison is introduced later.**
  – other DBMS families appear in the final modern database lecture and in projects



MySQL architecture

Figure 18.3 MySQL Architecture with Pluggable Storage Engines



[MySQL 8.4 Reference Manual, §18.11, Fig.18.3]

Storage engines: same SQL layer, different physical behavior

• **A storage engine is responsible for physical data storage and access.**

– engines can differ in transactions, locking, indexing and persistence behavior

– applications normally interact through the common SQL/server layer

• **InnoDB is the main engine for this course.**

– default transactional storage engine in modern MySQL

– supports transactions, row-level locking and foreign keys

– used in labs on transactions, locking, indexing and recovery

• **Other engines may be observed, not studied deeply.**

– Memory, MyISAM and others can be introduced through SHOW ENGINES

– engine differences are useful for understanding that physical implementation matters

Engine	Useful observation in L1
InnoDB	transactions, row-level locking, foreign keys
MyISAM	non-transactional; historical/legacy context
MEMORY	stored in memory; data lost on restart
NDB Cluster	distributed/cluster engine; introduced later

```
SHOW ENGINES;
```


[MySQL 8.4 Reference Manual, §18.11; MySQL 8.4 Manual, Storage Engines]

MySQL metadata inspection: what the DBMS knows about itself

- **INFORMATION_SCHEMA** provides access to metadata.
  – schemas, tables, columns, constraints, routines, statistics and more
  – useful for administration, debugging and understanding DBMS state
- **SHOW** statements provide quick inspection commands.
  – SHOW DATABASES, SHOW TABLES, SHOW COLUMNS
  – SHOW INDEX, SHOW ENGINES, SHOW VARIABLES, SHOW PROCESSLIST
- **Lab 0 emphasis**
  – students should separate queries that inspect user data from queries that inspect the DBMS

```
SELECT table_schema, table_name, engine,
table_rows
FROM information_schema.tables
WHERE table_schema = 'sakila';
```

Column	What it tells us
table_schema	which database/schema owns the table
table_name	name of each table
engine	storage engine used by the table
table_rows	estimated number of rows for some engines

[MySQL 8.4 Reference Manual, INFORMATION_SCHEMA; SHOW statements]

Basic MySQL inspection commands for Lecture 1

Command	What it checks	Why it matters
SELECT VERSION();	server version	verify environment and report reproducibility
SHOW DATABASES;	available databases	distinguish user schemas from system schemas
USE sakila; SHOW TABLES;	tables in sample database	confirm sample data was loaded
SHOW ENGINES;	supported storage engines	connect MySQL product behavior to storage-engine idea
SHOW PROCESSLIST;	active sessions/commands	show that DBMS is a running multi-session system
SHOW VARIABLES LIKE "%version%";	server configuration variables	introduce configuration/administration perspective
SELECT ... FROM information_schema.tables;	metadata about tables	connect catalog/data dictionary to real inspection

[MySQL 8.4 Reference Manual, SHOW statements; INFORMATION_SCHEMA]

From book concepts to MySQL objects

Book concept	MySQL object/command	Used later in
Database engine	mysqld + storage engines	L1, L3, L4
Data dictionary / catalog	INFORMATION_SCHEMA, data dictionary	L1, L7, L11
Storage manager	InnoDB files, buffer pool, tablespaces	L3
Query processor	EXPLAIN / optimizer / execution plan	L5, L11
Transaction manager	START TRANSACTION, COMMIT, ROLLBACK, locks	L6–L7
Authorization manager	CREATE USER, GRANT, REVOKE, roles	L8
Recovery/logging	backup, binary log, InnoDB redo/undo concepts	L9

This mapping helps students use a product without losing the general DBMS concepts.

[SKS 7e, Ch.1; MySQL 8.4 Reference Manual]

Lab 0 — Setup and inspection of a running DBMS

Lab objective

- Install or access a MySQL 8.x server.
- Load or connect to the **Sakila sample database**.
- Run inspection commands that reveal server version, schemas, engines, metadata and active sessions.
- **Submit a short lab report** with commands, outputs and observations.

Lab output

Evidence	Minimum expectation
Environment	server version and client/tool used
Sample DB	screenshot/output confirming Sakila loaded
Metadata	query against INFORMATION_SCHEMA
Processes	SHOW PROCESSLIST output
Reflection	3–5 sentences explaining database vs DBMS

[MySQL 8.4 Reference Manual, Ch.2 Installing MySQL; Sakila Sample Database]

Lab 0.1 — Installation / access requirement

- **Local installation**
  - install MySQL Community Server 8.x
  - install a SQL client such as MySQL Workbench, DBeaver or MySQL Shell
  - record the version used in the report
- **Required official references**
  - MySQL 8.4 Manual — Chapter 2 Installing MySQL
  - MySQL Sakila Sample Database documentation

Official links to include in lab handout

```
MySQL installation:
https://dev.mysql.com/doc/refman/8.4/en/installing.html

Sakila sample database:
https://dev.mysql.com/doc/sakila/en/
```


[MySQL 8.4 Reference Manual, Ch.2; MySQL Sakila Sample Database]

Lab 0.2 — Load and verify the sample database

- **Dataset**
  – Sakila sample database is used because many existing course labs already use it
  – later lectures reuse the same schema for views, procedures, triggers, indexing and backup
- **Verification tasks**
  – confirm database exists
  – list tables
  – inspect at least one table definition
  – run one simple SELECT query
- **Report requirement**
  – include command and output for each verification step

```
SHOW DATABASES;
USE sakila;
SHOW TABLES;
DESCRIBE film;
SELECT COUNT(*) FROM film;
```

Do not submit only screenshots of the GUI. Include SQL commands and copied outputs.

[MySQL Sakila Sample Database; MySQL 8.4 Reference Manual, SHOW statements]

Lab 0.3 — Inspect the DBMS, not only the data

- **Part A: server and engines**
  - What MySQL version is running?
  - Which storage engines are supported?
  - Which engine is used by Sakila tables?
- **Part B: metadata**
  - How many tables are in Sakila?
  - What columns does a chosen table contain?
  - What indexes exist on that table?
- **Part C: sessions/processes**
  - What sessions are connected?
  - What command is each session currently running?

```
SELECT VERSION();
SHOW ENGINES;

SELECT table_name, engine, table_rows
FROM information_schema.tables
WHERE table_schema = 'sakila';

SHOW PROCESSLIST;
```


[MySQL 8.4 Reference Manual, INFORMATION_SCHEMA; SHOW ENGINES; SHOW PROCESSLIST]

Lab 0.4 — Submission checklist

Item	What to submit	Pass condition
Environment	version, OS/tool, connection method	version and tool are clear
Sample database	evidence that Sakila is loaded	SHOW DATABASES/SHOW TABLES output included
Server inspection	VERSION, ENGINES, VARIABLES or equivalent	at least three DBMS-level outputs
Metadata inspection	INFORMATION_SCHEMA query output	query text and result included
Live-system view	SHOW PROCESSLIST or equivalent	output captured and explained
Reflection	short answer: database vs DBMS vs database system	technically correct and specific

Suggested lab-report length: 2–4 pages. Focus on evidence and explanation, not screenshots only.

What to remember from Lecture 1

• A DBMS is a software system for controlled management of data.

– it provides efficient access, integrity, transactions, security and recovery

• The database engine has several cooperating subsystems.

– query processor, storage manager, transaction/recovery, authorization/integrity, catalog

• SQL is an interface; execution requires system decisions.

– validation, plan selection, storage access, buffering and correctness checks

• MySQL is our implementation platform.

– concepts come from the textbook; product behavior comes from the MySQL manual

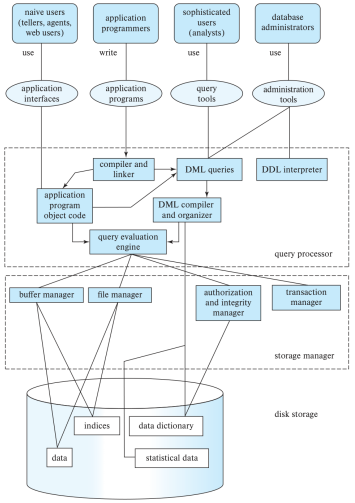


Figure 1.3 System structure.

Figure 1.3 — System structure. Source: SKS, Database System Concepts, 7e, Ch.1, p.22

Reading and references for Lecture 1

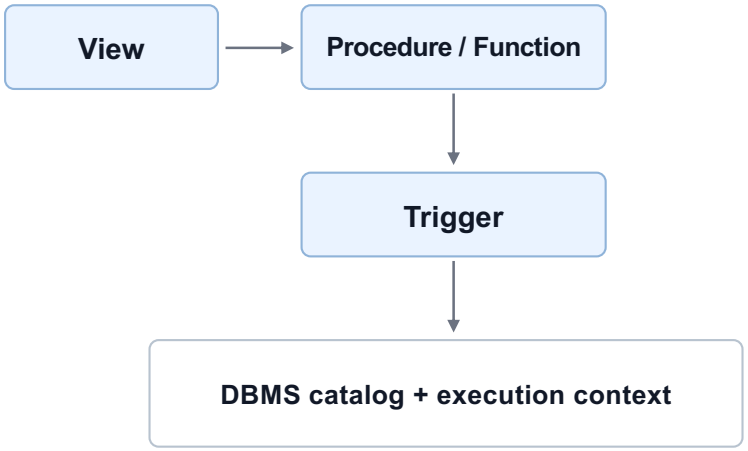
- **Required reading**
  – Silberschatz, Korth & Sudarshan, Database System Concepts, 7th ed., Chapter 1: §1.1–§1.8
  – Focus on: purpose of DBMS, view of data, database languages, database engine, application architecture, users/administrators
- **Implementation reference**
  – MySQL 8.4 Reference Manual: Chapter 2 Installing MySQL; INFORMATION_SCHEMA; SHOW statements; §18.11 Figure 18.3 MySQL Architecture with Pluggable Storage Engines

Preview of Lecture 2

Next key problem

What logic should live inside the database system instead of the application?

- Views for abstraction and controlled access.
- Stored procedures and functions for server-side logic.
- Triggers for automatic reactions to data changes.
- Trade-offs: reuse/security versus hidden logic and portability.



[SKS 7e, Ch.4–5; MySQL 8.4 Reference Manual, Stored Objects]